

# **RISC-V VeeR EL2 Based Real-Time Data Buffering SoC**

**Name : P Ganesh**

**Roll Number : 1602-23-735-317**

## **1. Problem Statement**

Modern embedded and System-on-Chip (SoC) systems frequently receive data streams from peripheral devices that must be temporarily stored and transferred to memory while the processor performs other tasks. When incoming data arrives faster than the processor can handle, directly transferring every data element through software can increase processor workload and may result in data loss or inefficient processor utilization. Therefore, a hardware-assisted data buffering and transfer mechanism is required to reliably manage incoming data and reduce unnecessary processor involvement.

This project proposes the design and verification of a RISC-V VeeR EL2 based SoC with dedicated hardware IPs for data buffering, direct memory transfer, and interrupt management. The SoC integrates the provided VeeR EL2 RISC-V processor with instruction memory, data memory, bus interconnect, UART, Timer, and GPIO as the mandatory system components. Three additional IP blocks—a FIFO, DMA Controller, and Interrupt Controller—are integrated into the SoC as memory-mapped peripherals. The processor executes an application that configures the data-transfer subsystem, monitors transfer status, and reports the received and transferred data information through UART.

## **2. Proposed SoC Architecture**

The proposed SoC is organized around the VeeR EL2 RISC-V processor as the central processing unit. The VeeR EL2 is a 32-bit RV32IMC processor with a four-stage scalar in-order pipeline. The processor communicates with instruction memory, data memory, and peripheral IPs through the system interconnect, while the VeeR EL2 core complex supports configurable system-bus interfaces according to the provided processor RTL and project requirements.

Instruction Memory stores the application program executed by the processor, while Data Memory stores application data and transferred data blocks. The bus interconnect performs address decoding and routes processor read and write transactions to the appropriate memory or peripheral based on the assigned memory map. UART is used to report data-transfer status and system information, Timer provides timing functionality, and GPIO provides general-purpose input/output control.

The additional data-transfer IPs operate as follows:

- **Data Buffering:** The FIFO temporarily stores incoming data so that data can be retained while the DMA Controller performs the transfer. FIFO status signals such as empty, full, and data count are used to manage data availability and overflow conditions.

- **Direct Memory Transfer:** The DMA Controller transfers data from the FIFO to Data Memory without requiring the processor to handle every individual data word. The processor configures the source, destination, transfer length, and control registers before the transfer begins.
- **Interrupt Management:** The Interrupt Controller monitors events such as DMA transfer completion, transfer errors, and FIFO conditions. It generates an interrupt to the RISC-V processor when a programmed event requires processor attention.

The overall data flow is:

**Data Input → FIFO → DMA Controller → Data Memory → RISC-V Processor → UART**

The Interrupt Controller operates alongside the data-transfer path to notify the processor of important transfer events. The processor configures and monitors the additional IPs through their memory-mapped control and status registers. This architecture demonstrates hardware/software interaction, FIFO-based buffering, direct memory access, interrupt-driven control, memory-mapped peripheral integration, and SoC-level RTL design.

### **3. Selected IPs:**

#### **3.1 FIFO**

The FIFO is a reusable data-buffering IP used to temporarily store incoming data. It provides controlled write and read operations and maintains status information such as full, empty, and data count conditions. The FIFO helps prevent data loss when incoming data cannot be transferred to memory immediately.

#### **3.2 DMA Controller**

The DMA Controller is a hardware IP used to transfer data between the FIFO and Data Memory with minimal processor involvement. It provides configurable source address, destination address, and transfer length registers. After configuration by the RISC-V processor, the DMA Controller performs the programmed data transfer and provides transfer-completion status.

#### **3.3 Interrupt Controller**

The Interrupt Controller is a lightweight hardware IP used to collect and manage interrupt requests generated by the DMA Controller and FIFO. It provides interrupt-enable, pending, and clear control and forwards valid interrupt requests to the RISC-V processor. The processor can access the interrupt status through memory-mapped registers.

The completed SoC will be verified through RTL simulation using normal data transfers, multiple data buffering, FIFO-full and empty conditions, different DMA transfer lengths, DMA transfer completion, and interrupt-generation scenarios. The processor application will configure the FIFO and DMA Controller, monitor transfer status, service interrupts, and report the results through UART.